DOMAIN DRIVEN DESIGN - JAVA - SPRINT 4

SellerFit

Documentação do Projeto

Equipe SellerFitDevs

Mateus Saavedra De Mendonça Cruz - RM563266

Davi Melo - RM561818

Danilo Fernandes - RM561657

Arthur Berlofa - RM564438

Ulisses Ribeiro - RM562230

FIAP - Engenharia de Software - Challenge 2026

Sumário

1. Objetivo e Escopo do Projeto
2. Principais Funcionalidades
3. Tabela de Endpoints
4. Protótipo - Telas
5. Modelo do Banco de Dados
6. Diagrama de Classes
7. Como Executar

1. Objetivo e Escopo do Projeto

O SellerFit ajuda o time comercial da TOTVS depois de uma reunião com o cliente. O vendedor cola a transcrição da conversa e o sistema aponta as dores do cliente, se existe risco de cancelamento (churn) ou chance de venda (upsell) e qual atendente da TOTVS é o mais indicado para continuar o atendimento.

Na Sprint 3 essa lógica rodava em Java puro pelo console. Nesta sprint o projeto virou uma API Restful em Spring Boot, ligada ao banco Oracle da FIAP e consumida pelo front-end em Next.js. A leitura da transcrição passou a ser feita pela API da OpenAI.

2. Principais Funcionalidades

- **Análise da reunião:** identifica as dores do cliente, a urgência e o perfil a partir da transcrição.
- **Sinal de negócio:** cruza a nota NPS com o que foi falado (concorrentes, reclamações, interesse em comprar) para indicar churn, upsell ou neutro.
- **Match de atendentes:** dá uma nota de 0 a 100 para cada atendente, considerando perfil, segmento, foco e avaliação, e sugere os três melhores.
- **Integração com a OpenAI:** a IA lê a transcrição e recomenda um atendente com justificativa. Sem a chave configurada, o sistema usa regras próprias por palavra-chave e continua funcionando.
- **CRUD completo** de clientes, atendentes, reuniões, dores e matches, com validações nos services e tratamento de erros na API.

3. Tabela de Endpoints

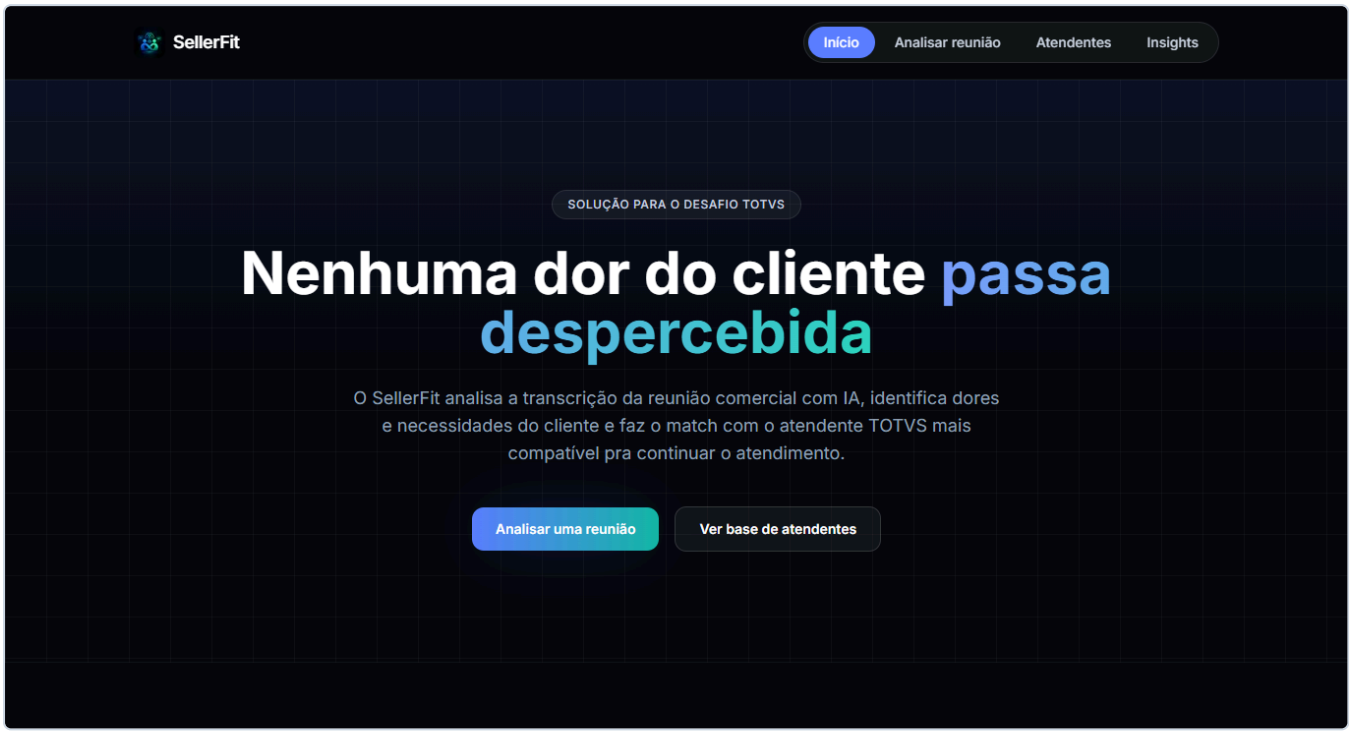
Base: http://localhost:8080. Todas as requisições e respostas usam JSON.

Verbo	URI	Descrição	Status
Análise da reunião			
POST	`/api/analises`	Analisa a transcrição e sugere os atendentes	201, 400, 500
GET	`/api/analises/{idReuniao}`	Consulta uma análise feita	200, 400, 404, 500
GET	`/api/health`	Verifica se a API está no ar	200
Clientes			
GET	`/api/clientes`	Lista todos os clientes	200, 500
GET	`/api/clientes/{id}`	Busca um cliente pelo id	200, 400, 404, 500
GET	`/api/clientes/{id}/reunioes`	Histórico de reuniões do cliente	200, 400, 500
POST	`/api/clientes`	Cadastra um cliente	201, 400, 500
PUT	`/api/clientes/{id}`	Atualiza um cliente	200, 400, 404, 500
DELETE	`/api/clientes/{id}`	Exclui um cliente	204, 400, 404, 500
Atendentes			
GET	`/api/atendentes`	Lista todos os atendentes	200, 500
GET	`/api/atendentes/{id}`	Busca um atendente pelo id	200, 400, 404, 500
POST	`/api/atendentes`	Cadastra um atendente	201, 400, 500
PUT	`/api/atendentes/{id}`	Atualiza um atendente	200, 400, 404, 500
DELETE	`/api/atendentes/{id}`	Exclui um atendente	204, 400, 404, 500
Reuniões			
GET	`/api/reunioes`	Lista todas as reuniões	200, 500
GET	`/api/reunioes/{id}`	Busca uma reunião pelo id	200, 400, 404, 500
GET	`/api/reunioes/{id}/dores`	Dores identificadas na reunião	200, 400, 500
GET	`/api/reunioes/{id}/matches`	Matches gerados para a reunião	200, 400, 500
POST	`/api/reunioes`	Cadastra uma reunião	201, 400, 500
PUT	`/api/reunioes/{id}`	Atualiza uma reunião	200, 400, 404, 500
DELETE	`/api/reunioes/{id}`	Exclui uma reunião	204, 400, 404, 500
Dores			
GET	`/api/dores`	Lista todas as dores	200, 500
GET	`/api/dores/{id}`	Busca uma dor pelo id	200, 400, 404, 500
POST	`/api/dores`	Cadastra uma dor	201, 400, 500
PUT	`/api/dores/{id}`	Atualiza uma dor	200, 400, 404, 500
DELETE	`/api/dores/{id}`	Exclui uma dor	204, 400, 404, 500
Matches			
GET	`/api/matches`	Lista todos os matches	200, 500
GET	`/api/matches/{id}`	Busca um match pelo id	200, 400, 404, 500
POST	`/api/matches`	Cadastra um match	201, 400, 500
PUT	`/api/matches/{id}`	Atualiza um match	200, 400, 404, 500
DELETE	`/api/matches/{id}`	Exclui um match	204, 400, 404, 500

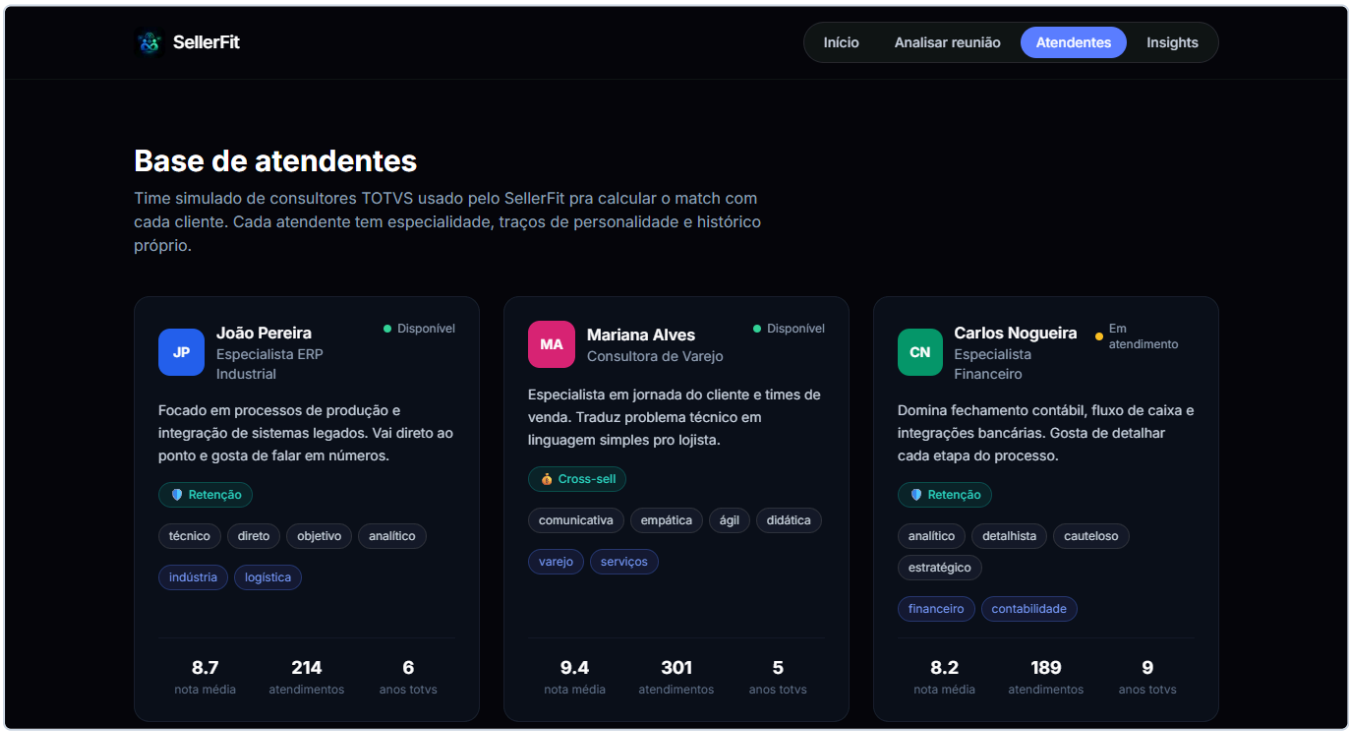
200 sucesso, **201** criado, **204** excluído, **400** dados inválidos, **404** não encontrado, **500** erro no banco.

4. Protótipo - Telas

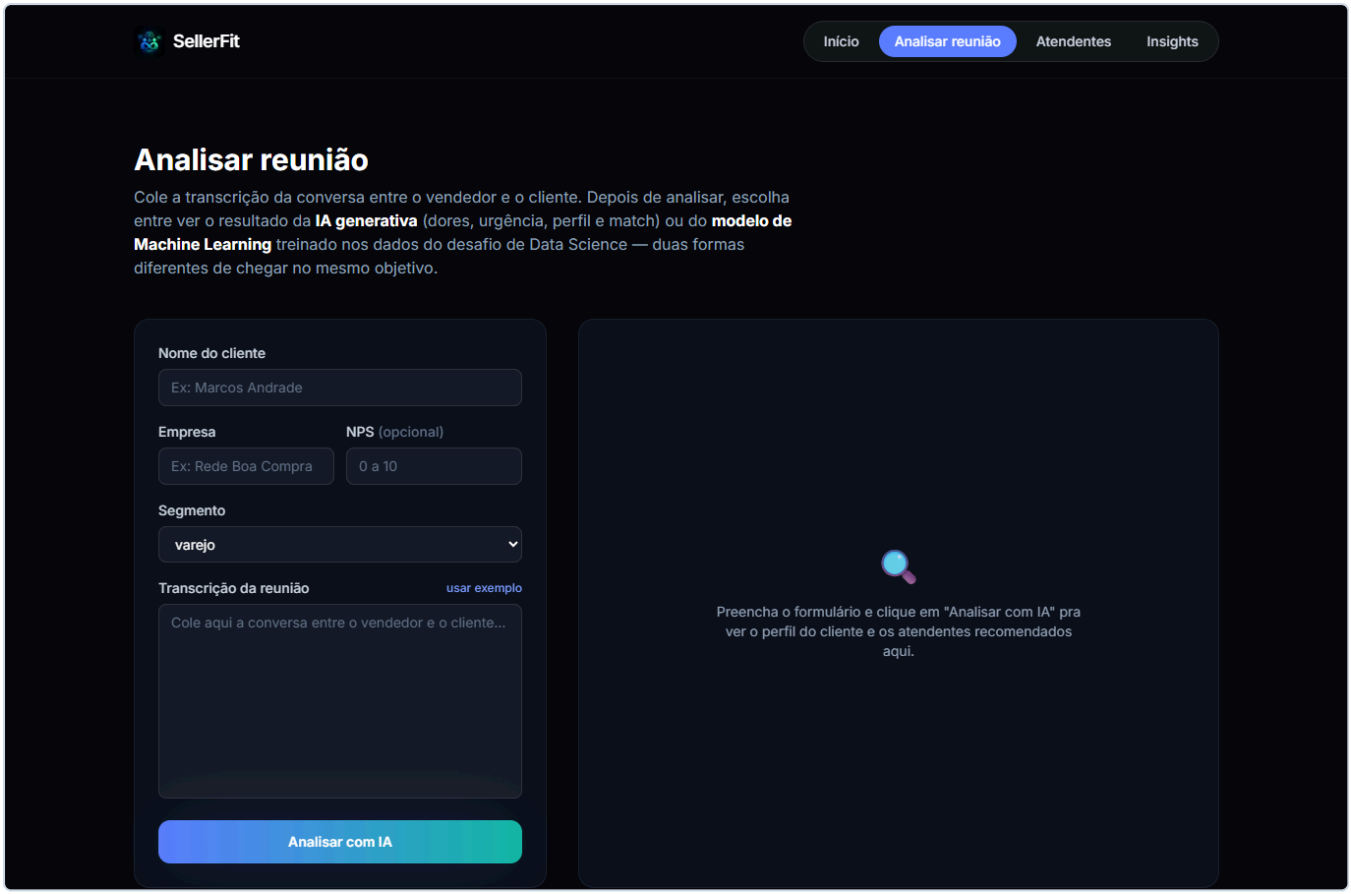
- Página inicial



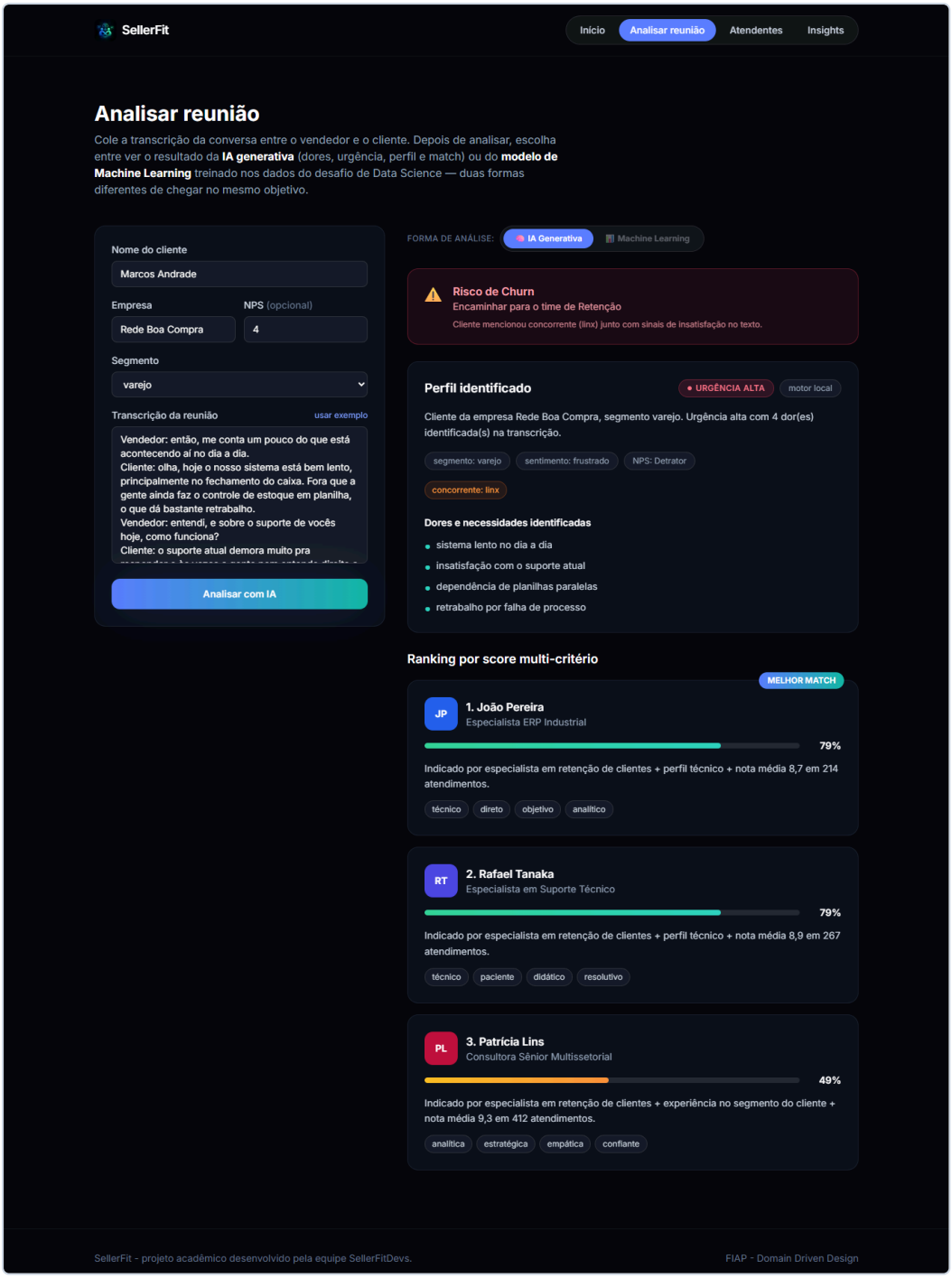
- Base de atendentes



- Analisar reunião

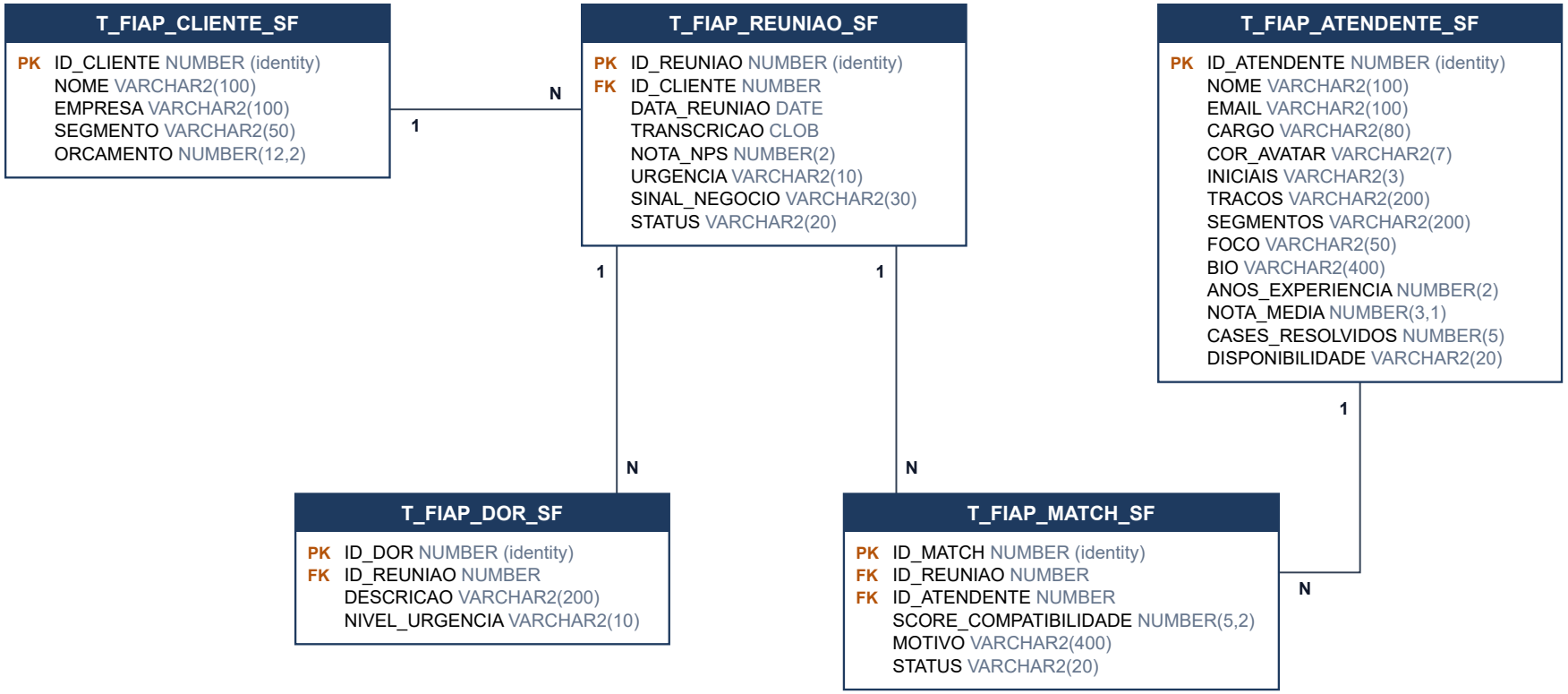


- Resultado da análise



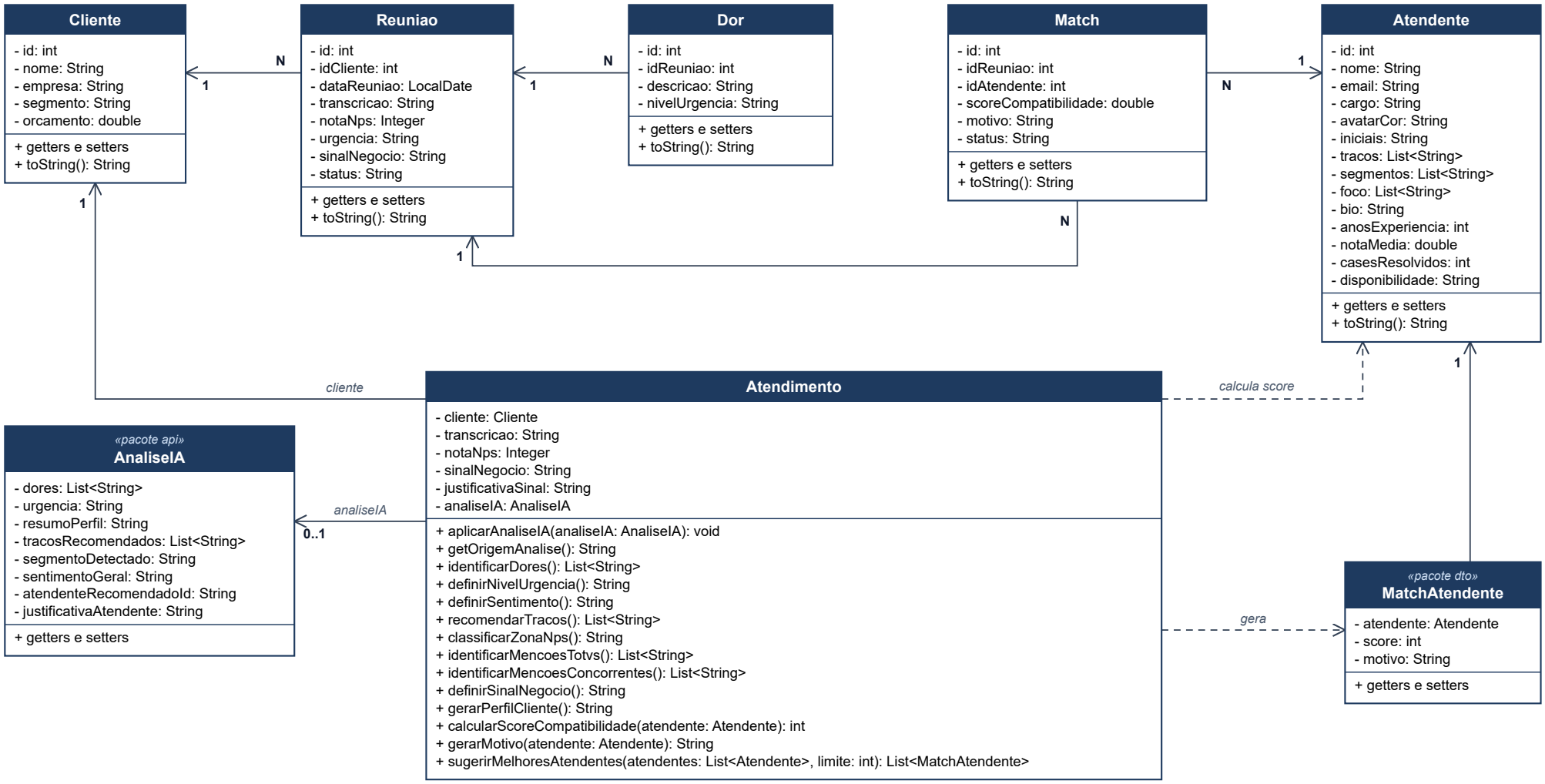
5. Modelo do Banco de Dados

Cinco tabelas no Oracle. Um cliente tem várias reuniões, cada reunião tem suas dores e seus matches, e cada match liga a reunião a um atendente.

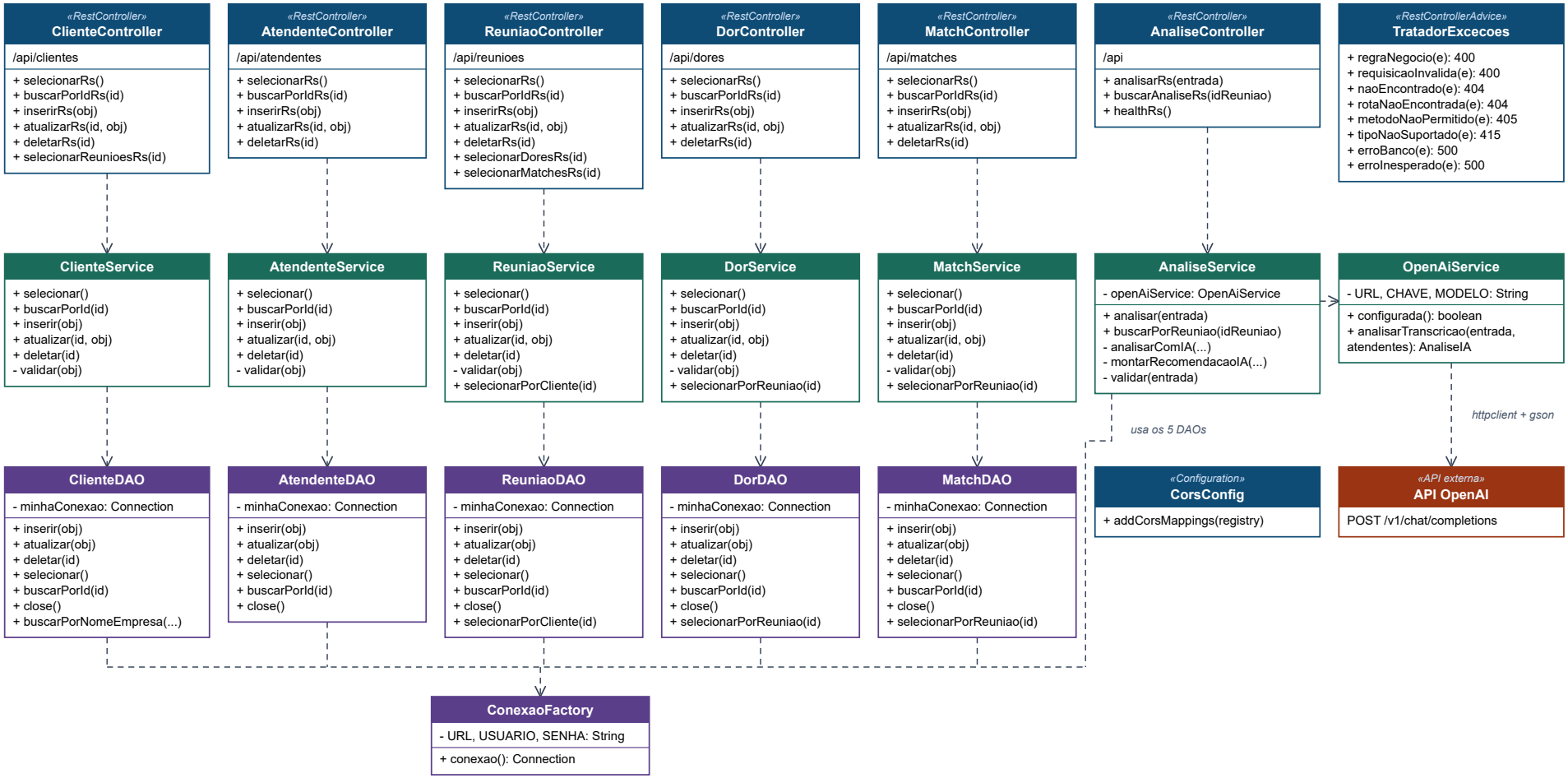


6. Diagrama de Classes

- Camada Model



- Camadas Controller, Service e DAO



7. Como Executar

- Banco: as tabelas já estão criadas no Oracle da FIAP. O script de criação fica no arquivo **script.sql**.
- Configuração: usuário e senha do Oracle e a chave da OpenAI não ficam no código. Localmente vão no arquivo **local.properties** e no servidor em variáveis de ambiente.
- API: na pasta do projeto, rodar **.\mvnw spring-boot:run** (ou a classe SellerfitApplication no IntelliJ). Ela sobe em http://localhost:8080.
- Publicação: a API roda no Render usando o Dockerfile do projeto. O front-end fica em um projeto separado, publicado na Vercel.